

Hotelería / Turismo

Hotel Costa Serena

1. Contexto del caso

El **Hotel Costa Serena** requiere un sistema para administrar las habitaciones que ofrece y registrar la tarifa por noche de cada una. El hotel trabaja con distintos tipos de habitaciones; aunque todas comparten información básica como número de habitación, piso y capacidad de huéspedes, la tarifa varía según el tipo de habitación.

Adicionalmente, ciertas habitaciones son candidatas a un servicio de salida tardía (late checkout), lo que implica gestionar su disponibilidad de forma diferenciada. El sistema debe modelar esta situación aplicando los principios de la Programación Orientada a Objetos, de manera que el comportamiento compartido y el comportamiento específico de cada tipo de habitación queden correctamente organizados en la jerarquía de clases.

2. Información necesaria

Cada clase debe incluir constructor(es), métodos accedadores (getters), mutadores (setters) y el método `toString()`, el cual debe incluir únicamente el número de habitación y el piso. Los constructores deben delegar la asignación de valores a los métodos `setter`, de modo que las validaciones definidas se apliquen desde la construcción del objeto.

- Todas las habitaciones del sistema comparten número de habitación, piso y capacidad de huéspedes. Dado que la tarifa se determina de manera diferente para cada tipo de habitación, el sistema debe contemplar un mecanismo que permita calcular esa tarifa de forma general sin conocer el tipo específico del objeto en cada momento.
- Las **habitaciones suite** agregan el tipo de vista (mar, ciudad, montaña, etc.), si incluyen desayuno buffet y un indicador de late checkout activo. Al participar en el servicio de late checkout del hotel, deben implementar el contrato definido por la interfaz **ConLateCheckout**.
- Las **habitaciones estándar** agregan un atributo que indica la cantidad de camas adicionales, condición que incide en la tarifa de la habitación.
- El gestor del hotel es la clase responsable de administrar la colección de habitaciones registradas y exponer las operaciones sobre ella.

3. Métodos especializados

El sistema debe ser capaz de calcular la tarifa para cualquier habitación registrada, independientemente del tipo. Cada tipo de habitación determina esa tarifa según su propia lógica:

- **Habitación Suite:** la tarifa base es \$95.000. Si la habitación no incluye desayuno buffet, esa tarifa se incrementa en un 10%.
- **Habitación Estándar:** la tarifa base es \$55.000. Si la cantidad de camas adicionales supera 1, la tarifa se incrementa en un 20%.

Interfaz ConLateCheckout

Define el contrato que deben cumplir las habitaciones que participan en el servicio de salida tardía del hotel. Está compuesta por dos operaciones:

- La primera permite consultar en cualquier momento si la habitación tiene late checkout activo.
- La segunda permite activar el late checkout de la habitación.

Esta interfaz está diseñada de manera que, en el futuro, cualquier tipo de habitación pueda incorporarse al servicio de late checkout sin necesidad de modificar la jerarquía de clases existente. Por ahora, solo las habitaciones suite implementan este contrato.

4. Validaciones esperadas

Las validaciones deben aplicarse en los métodos setter de cada clase. Cuando una condición no se cumpla, debe lanzarse una excepción de tipo `IllegalArgumentException` con un mensaje descriptivo que explique el motivo del rechazo.

- **número de habitación:** no puede ser nulo ni vacío.
- **piso:** debe encontrarse en el rango entre 1 y 20.
- **capacidad de huéspedes:** debe ser un valor mayor que cero.

5. Operaciones del sistema

- Registrar una habitación en la colección e informar por consola que fue incorporada correctamente.
- Buscar y retornar las habitaciones cuyo número coincida con el criterio de búsqueda recibido.

6. Requerimientos de la clase principal

- Instanciar una habitación de cada tipo disponible utilizando los datos de la tabla siguiente.
- Marcar la Habitación Suite '**S-301**' con late checkout activo una vez creada.
- Registrar todas las habitaciones en el gestor del sistema.
- Invocar las operaciones del gestor y mostrar sus resultados: buscar por número y listar todas las habitaciones registradas mediante el método toString() de cada objeto.

Datos para la clase principal

Tipo	Número	Piso	Capacidad	Atributo específico
Habitación Suite	S-301	3	2	Vista: Mar Desayuno buffet: No
Habitación Suite	S-402	4	3	Vista: Ciudad Desayuno buffet: Sí
Habitación Estándar	E-101	1	2	Camas adicionales: 2
Habitación Estándar	E-102	1	2	Camas adicionales: 0

7. Resultado esperado

La ejecución del programa con los datos indicados debe producir una salida que muestre, para cada habitación, su información completa (incluyendo la tarifa calculada), el resultado de la búsqueda por número y el listado completo mediante toString(). Se aceptan pequeñas diferencias en los textos o en el formato de los números, siempre que se muestren todos los datos solicitados y que los resultados numéricos sean correctos.

Guía de práctica generada como material de estudio. No corresponde a una evaluación oficial.